CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The Tata CLiQ E-Commerce Database is designed to efficiently manage customers, sellers, brands, categories, products, carts, orders, payments, shipments, reviews, and wishlists. Functional dependencies identify the relationships between attributes in each table, while redundancy analysis and data anomaly detection ensure data consistency and eliminate unnecessary duplication. A well-designed database improves performance, maintains data integrity, and supports efficient management of e-commerce operations.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → First_Name, Last_Name, Email, Phone, Password, Address

Redundancy Analysis

Customer information is stored only once in the Customer table. Other tables reference the customer using **Customer_ID**, thereby preventing duplicate customer records.

Data Anomalies

- **Insert Anomaly:** A new customer can be added without placing an order.
- **Update Anomaly:** Updating customer information requires changes only in the Customer table.
- **Delete Anomaly:** Deleting an order will not remove customer information.

2. SELLER TABLE

Functional Dependency

Seller_ID → Seller_Name, Email, GST_Number

Redundancy Analysis

Seller information is maintained only once. Products reference sellers using **Seller_ID**, avoiding duplicate seller details.

Data Anomalies

- **Insert Anomaly:** A seller can be added before listing products.
- **Update Anomaly:** Seller details are updated only in the Seller table.
- **Delete Anomaly:** Deleting a product will not remove seller information.

3. BRAND TABLE

Functional Dependency

Brand_ID → Brand_Name

Redundancy Analysis

Each brand is stored only once. Products reference the brand through **Brand_ID**, eliminating repeated brand names.

Data Anomalies

- **Insert Anomaly:** A brand can be created without products.
- **Update Anomaly:** Brand name changes are made only once.
- **Delete Anomaly:** Deleting a product does not delete the brand.

4. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name

Redundancy Analysis

Category information is stored only once. Products reference categories using **Category_ID**, preventing duplicate category names.

Data Anomalies

- **Insert Anomaly:** Categories can be added before products.
- **Update Anomaly:** Category names are updated only once.
- **Delete Anomaly:** Deleting a product does not remove category information.

5. PRODUCT TABLE

Functional Dependency

Product_ID → Brand_ID, Category_ID, Seller_ID, Price, Stock

Redundancy Analysis

Product information is stored only in the Product table. Other tables use **Product_ID** as a reference instead of repeating product details.

Data Anomalies

- **Insert Anomaly:** Products can be added before customers place orders.
- **Update Anomaly:** Product price and stock are updated only once.
- **Delete Anomaly:** Deleting an order does not remove product information.

6. CART TABLE

Functional Dependency

Cart_ID → Customer_ID

Redundancy Analysis

Cart information is maintained separately and linked to customers using **Customer_ID**, preventing duplicate cart records.

Data Anomalies

- **Insert Anomaly:** A cart can be created only for an existing customer.
- **Update Anomaly:** Cart details are updated only once.
- **Delete Anomaly:** Deleting a cart does not affect customer information.

7. ORDER TABLE

Functional Dependency

Order_ID → Customer_ID

Redundancy Analysis

The Order table stores only order-related information. Customer details are accessed through **Customer_ID**, minimizing redundancy.

Data Anomalies

- **Insert Anomaly:** Orders can be created only for registered customers.
- **Update Anomaly:** Order information is updated only once.
- **Delete Anomaly:** Deleting an order does not remove customer information.

8. ORDER_ITEM TABLE

Functional Dependency

OrderItem_ID → Order_ID, Product_ID

Redundancy Analysis

This table connects orders and products using foreign keys, preventing repeated product information.

Data Anomalies

- **Insert Anomaly:** Order items can be added only after creating an order.
- **Update Anomaly:** Changes affect only the selected order item.

- **Delete Anomaly:** Removing an order item does not delete product details.

9. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID

Redundancy Analysis

Payment information is stored separately from the Order table to avoid duplicate payment records.

Data Anomalies

- **Insert Anomaly:** Payment details are recorded after an order is created.
- **Update Anomaly:** Payment information is updated only once.
- **Delete Anomaly:** Deleting payment details does not remove the order.

10. SHIPMENT TABLE

Functional Dependency

Shipment_ID → Order_ID

Redundancy Analysis

Shipment details are stored separately and linked through **Order_ID**, preventing repeated shipment information.

Data Anomalies

- **Insert Anomaly:** Shipment records can be created after order confirmation.
- **Update Anomaly:** Shipment details are updated independently.
- **Delete Anomaly:** Deleting shipment information does not affect order records.

11. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID

Redundancy Analysis

Review information is stored separately. Customer and product references avoid duplication of review-related data.

Data Anomalies

- **Insert Anomaly:** Reviews can be added only by existing customers for existing products.

- **Update Anomaly:** Review information is updated only once.
- **Delete Anomaly:** Deleting a review does not affect customer or product information.

12. WISHLIST TABLE

Functional Dependency

Wishlist_ID → Customer_ID, Product_ID

Redundancy Analysis

Wishlist records are maintained separately using foreign keys, preventing duplicate customer and product information.

Data Anomalies

- **Insert Anomaly:** Wishlist items can be added only for existing customers and products.
- **Update Anomaly:** Wishlist information is updated only once.
- **Delete Anomaly:** Removing a wishlist item does not delete customer or product records.

4.2 CONCLUSION

The Tata CLiQ E-Commerce Database is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant information while maintaining relationships through primary and foreign keys. This design minimizes redundancy, prevents insertion, update, and deletion anomalies, ensures data integrity, and supports efficient, secure, and scalable e-commerce database management.